



**FACULTY OF BUSINESS SCIENCES DEPARTMENT OF INFORMATION AND
MARKETING SCIENCES**

NAME & REG NUM : BURTON MAREKE (R2514163F)

PROGRAM : M COMM INFORMATION SYSTEM MANAGEMENT

COURSE CODE : MIM 736 SOFTWARE ENGINEERING (PRACTICAL)

LEVEL : 1.2

LECTURER : Dr. M. ZHOU

1. Executive Summary

I contributed to the backend of the Qualification Verification System by implementing and maintaining core verification logic, supporting authentication and qualification workflows, integrating H2 and PostgreSQL databases, strengthening automated test coverage, and completing a focused code-cleaning pass. The work improved reliability, maintainability, and readiness for both local development and Docker-based deployment without changing the intended application behaviour.

2. Backend Logic Contribution

My backend contribution covered the main Spring Boot application layers:

- Verification workflow: qualification lookup by verification code, credential ID, or candidate name; credential hash validation; revoked and expired qualification handling; and audit record creation.
- Qualification and identity services: management of qualification records, users, institutions, roles, student stages, credential identifiers, and credential hashes.
- Security: JWT-based authentication support, user-detail loading, role-aware access, and protected service/controller flows.
- Agentic insights: local rule-based verification analytics, risk detection, recommendations, trend calculations, and optional LLM-assisted summaries.
- Demo data: repeatable seeding of institutions, users, students, qualifications, statuses, and verification history for development and demonstrations.

3. Database Integration

The backend supports two database modes. H2 is configured for local file-based development, while the Docker profile uses PostgreSQL with environment-based connection settings. Hibernate manages schema updates, and dedicated schema aligners address legacy database differences that Hibernate does not always repair automatically.

- H2SchemaAligner detects H2 databases and widens legacy ENUM columns used by user roles and student stages.
- PostgresSchemaAligner detects legacy BYTEA qualification columns and converts them to the required text types.
- Docker configuration builds the backend with Java 21 and Maven, then runs it on a Java 21 runtime image.
- Database credentials, hosts, ports, JWT settings, CORS settings, and optional AI settings are externalised through configuration and environment variables.

4. Testing Contribution

I worked within the existing JUnit and Spring test structure and preserved testable service boundaries. The backend test suite includes:

- H2SchemaAlignerTest and PostgresSchemaAlignerTest for database compatibility behaviour.
- JwtServiceTest for token generation and validation behaviour.
- CredentialHashServiceTest for deterministic credential integrity hashing.
- QualificationServiceTest for qualification service operations.
- AgenticInsightsServiceTest for risk analysis, recommendations, and insight generation.
- QualificationVerificationIT for application-level qualification verification integration coverage.

5. Code Cleaning and Commit Highlights

I completed eight small, isolated, behaviour-preserving cleanup commits. Each commit was kept focused so it could be reviewed independently and reverted without affecting unrelated functionality:

- f4f5bbd - Clean up H2 schema constants.
- b9c0f77 - Clean up PostgreSQL schema constants.
- 8839533 - Clean up credential ID constants and generation bounds.
- 305a230 - Clean up verification method labels.
- ab54f9d - Clean up insight response constants.
- bf59b31 - Centralise featured seeded student usernames.
- b984dde - Make a stateless insight helper static.
- 0946f75 - Clean up the H2 column type constant.

6. Challenges and How They Were Solved

Challenge 1: Legacy database schemas did not always match the current domain model.

- Solution: Added focused H2 and PostgreSQL schema alignment components that detect the active database and apply only the required compatibility conversion.

Challenge 2: Verification required several possible lookup paths and different audit outcomes.

- Solution: Kept the lookup order explicit, calculated the canonical credential hash, evaluated status and expiry, and persisted an audit record for every verification attempt.

Challenge 3: The repository used both local H2 and Docker PostgreSQL environments.

- Solution: Preserved separate application profiles, externalised database settings, and added database-specific alignment logic instead of forcing one database-specific solution onto both environments.

Challenge 4: The local development shell did not have Java or Maven available.

- Solution: Used editor diagnostics to verify all touched Java files, kept changes small and type-safe, and relied on the repository Docker build configuration as the reproducible Maven build path.

Challenge 5: Individual pushes initially encountered GitHub authentication and permission errors.

- Solution: Investigated the Git state, confirmed each cleanup was committed separately, retried pushes individually, and verified that the commits were ultimately present on origin/main.

7. Lessons Learned

- Small commits make code review, troubleshooting, rollback, and contribution tracking much easier.
- Database migrations and schema compatibility need explicit handling when legacy databases are already in use.
- Verification systems must record both successful and unsuccessful attempts for auditability and security analysis.
- Environment-specific configuration should be externalised so local development and container deployment can share the same application code.
- Readability improvements such as named constants and clear ownership of static helpers reduce maintenance cost without introducing unnecessary abstractions.
- When a local toolchain is incomplete, a reproducible container build and focused static diagnostics provide useful validation, but a full Maven test run should still be completed in CI or an environment with Java and Maven installed.

8. Conclusion

The contribution strengthened the backend across functionality, data persistence, security-aware verification, testing structure, deployment compatibility, and maintainability. The final cleanup was delivered as eight traceable commits, with the working tree clean and the changes available on the main remote branch.